EJECUCIÓN DEL PROYECTO BIBLIOTECA-API CON DOCKER

Informe de práctica · Parte 2

Estudiante: ADRIAN MONTES STAVE

Código: T00081565

Asignatura: Arquitectura de Software

Actividad: Taller del 2.º corte — Semana 10

2026

Nota importante sobre las evidencias

Este informe está preparado como guía de documentación. Las capturas y salidas de terminal deben ser tomadas en el computador del estudiante después de ejecutar los pasos. No se incluyen capturas ficticias ni se afirma que los contenedores o los endpoints hayan funcionado sin comprobarlo. Antes de entregar, reemplaza los espacios de evidencia por capturas reales y ajusta la narración a lo que efectivamente ocurrió.

1. Objetivo y descripción

El objetivo de la práctica es ejecutar el proyecto biblioteca-api usando Docker y Docker Compose, verificar el estado de los servicios y probar el flujo de registro, autenticación y consulta de libros. Según el enunciado del taller, el proyecto usa Go, Gin y MySQL, con separación por capas handler → service → repository. La guía principal que se debe seguir es el README del repositorio: https://github.com/carrenoutb/biblioteca-api/blob/main/README.md

2. Fase 1 — Preparación e instalación

Primero se debe instalar Git y Docker Desktop (Windows/macOS) o Docker Engine y el complemento Docker Compose (Linux), según el sistema operativo. Luego se comprueba que ambos comandos estén disponibles:

```
docker --version\ndocker compose version\ngit --version
```

Evidencia requerida: captura de la terminal donde se vean las versiones obtenidas. Inserta aquí la imagen real: **[EVIDENCIA 1 — versiones de Docker y Compose]**.

3. Fase 2 — Clonar y reconocer el proyecto

En una terminal, se clona el repositorio y se entra en la carpeta:

```
git clone https://github.com/carrenoutb/biblioteca-api.git\ncd biblioteca-api
```

Antes de ejecutar, conviene revisar los archivos que describe la guía:

Dockerfile: contiene las instrucciones para construir la imagen de la API, como la imagen base, la copia del código y el comando de inicio. **docker-compose.yml:** describe los servicios que se ejecutan juntos, su configuración, red, puertos y posibles volúmenes o comprobaciones de salud. **.env.example:** sirve como plantilla de las variables de configuración; debe copiarse a `.env` y ajustarse si la guía lo exige. **migrations/:** contiene cambios versionados del esquema de base de datos para crear o actualizar las tablas de forma controlada.

Evidencia requerida: captura del listado de archivos y de la revisión de Dockerfile, docker-compose.yml, .env.example y migrations/. Inserta aquí: **[EVIDENCIA 2 — archivos del proyecto]**.

4. Fase 3 — Configuración del entorno

Se crea el archivo local de configuración copiando la plantilla:

```
cp .env.example .env
```

Las variables indicadas en el enunciado cumplen estas funciones generales; confirma los valores y el uso exacto en el README y en el archivo de Compose del repositorio:

Variable	Función general
DB_HOST	Nombre del host de la base de datos. En una red Compose suele ser el nombre del servicio, por ejemplo mysql.
DB_PORT	Puerto en el que la base de datos recibe conexiones dentro de la red de servicios; MySQL suele usar 3306.
DB_USER	Usuario que utiliza la API para conectarse a MySQL.
DB_PASSWORD	Contraseña del usuario de base de datos; debe coincidir con la configuración que usa MySQL.

Variable	Función general
DB_NAME	Nombre de la base de datos que utilizará la aplicación.
JWT_SECRET	Secreto utilizado para firmar o validar tokens JWT. Debe ser privado y suficientemente robusto.
JWT_EXPIRATION_HOURS	Tiempo de validez del token, expresado en horas.
APP_PORT	Puerto en el que la API escucha o que se configura para publicarla, según el proyecto.

No publiques el archivo .env con secretos reales. En un repositorio de entrega, verifica que esté ignorado por Git y comparte únicamente valores de ejemplo no sensibles.

Evidencia requerida: captura del comando ejecutado y del archivo de configuración con secretos ocultos. Inserta aquí: **[EVIDENCIA 3 — creación de .env; ocultar contraseñas y JWT_SECRET]**.

5. Fase 4 — Construcción y arranque

Desde la carpeta del proyecto, se ejecuta:

```
docker compose up --build
```

El comando construye las imágenes que lo necesiten y crea/inicia los servicios definidos en Compose. En la primera ejecución puede descargar imágenes base, instalar dependencias y construir la API; esto puede tardar según la conexión y el equipo. La terminal debe revisarse por si aparecen errores de construcción, conexión o configuración. Si todo queda iniciado, se puede abrir otra terminal para consultar el estado.

Evidencias requeridas: **[EVIDENCIA 4 — inicio de descarga/construcción]** y **[EVIDENCIA 5 — salida final con servicios iniciados]**. Las capturas deben mostrar la salida real.

6. Fase 5 — Verificación de servicios y API

Ejecuta:

```
docker compose ps
```

Verifica que los servicios estén activos y que MySQL alcance el estado saludable (healthy), si el archivo Compose define un healthcheck. Después abre en el navegador la dirección indicada por el taller: http://localhost:8080/swagger/index.html. Si no carga, revisa el mapeo de puertos, los logs y la configuración real del proyecto.

Evidencias requeridas: **[EVIDENCIA 6 — docker compose ps]** y **[EVIDENCIA 7 — Swagger abierto y respondiendo]**.

7. Fase 6 — Prueba del flujo de autenticación y libros

Siguiendo el tutorial de la sección 3.4 c) del README, usa REST Client de VS Code, Swagger UI o curl para ejecutar en orden: (1) registrar un usuario, (2) iniciar sesión, (3) copiar el token de autenticación devuelto, (4) incluirlo en la cabecera Authorization según el formato documentado y (5) solicitar el listado de libros. Deben guardarse las respuestas reales y ocultarse datos personales o secretos antes de compartir capturas.

Evidencias requeridas: **[EVIDENCIA 8 — registro exitoso]**, **[EVIDENCIA 9 — inicio de sesión y token oculto]** y **[EVIDENCIA 10 — consulta de libros autenticada]**.

8. Problemas comunes y posibles soluciones

Problema 1: el puerto ya está ocupado. Si Compose no puede publicar un puerto porque otra aplicación lo utiliza, identifica el proceso que lo ocupa o cambia el puerto del host en la configuración, manteniendo coherencia con la URL que se abre en el navegador. Luego vuelve a iniciar los servicios.

Problema 2: la API no conecta con MySQL. Revisa que DB_HOST use el nombre del servicio de Compose (por ejemplo, mysql) y no 127.0.0.1; comprueba que usuario, contraseña y nombre de base de datos coincidan y que MySQL esté saludable. Los logs se pueden consultar con `docker compose logs`.

Problema 3: Docker no está iniciado o Compose no se reconoce. Inicia Docker Desktop o el servicio Docker en Linux y verifica `docker --version` y `docker compose version`. Si el complemento falta, instala la versión compatible siguiendo la documentación oficial.

En la versión final del informe, indica cuáles de estos problemas ocurrieron realmente y qué solución aplicaste. Si no ocurrió ninguno, preséntalos como problemas potenciales, tal como permite el enunciado.

9. Preguntas de comprensión

1. ¿Qué diferencia hay entre una imagen y un contenedor? Usa un ejemplo de este proyecto.
Una imagen es la plantilla con el sistema de archivos y las instrucciones necesarias para ejecutar un servicio; un contenedor es una instancia creada a partir de esa imagen y que se encuentra ejecutándose o detenida. En biblioteca-api, la imagen de la API se construye usando el Dockerfile y, al iniciar Compose, se crea un contenedor para ejecutar el servidor. La imagen puede utilizarse para crear más de un contenedor.

2. ¿Qué hace docker compose que no harías con docker run por separado?
Compose permite describir y administrar varios servicios en un solo archivo: API, MySQL, redes, variables, puertos, volúmenes y comprobaciones de salud. Con `docker run` por separado habría que crear y mantener manualmente cada contenedor y coordinar sus parámetros y redes. `docker compose up --build` aplica la configuración declarada y arranca el conjunto de servicios.

3. ¿Por qué la API encuentra la base de datos con DB_HOST=mysql y no con 127.0.0.1?
Dentro de un contenedor, 127.0.0.1 apunta al propio contenedor de la API, no al contenedor de MySQL. Compose conecta los servicios a una red y normalmente ofrece resolución DNS por nombre de servicio; por eso la API puede usar `mysql` como nombre del servidor de base de datos. Esto depende de que el servicio realmente se llame así en el archivo Compose.

4. ¿Qué borra exactamente docker compose down -v y cuándo lo usarías?
Detiene y elimina los contenedores y las redes creadas por Compose; con `-v` también elimina los volúmenes nombrados declarados en el proyecto y los volúmenes anónimos asociados que Compose elimina con esa opción. Esto puede borrar los datos persistentes de MySQL. Lo usaría para reiniciar un entorno desechable desde cero, solo después de confirmar que no necesito conservar los datos. No es un comando rutinario de apagado.

5. ¿Qué pasaría si cambias DB_PASSWORD en .env pero no en docker-compose.yml? ¿Y al revés?
Depende de cómo Compose y el contenedor de MySQL consuman las variables. Si la API recibe una contraseña nueva pero MySQL sigue usando la anterior, la autenticación puede fallar. Si cambia la variable que configura MySQL pero la API conserva la contraseña antigua, también puede fallar la conexión. Además, las variables de inicialización de la imagen oficial de MySQL no necesariamente cambian la contraseña de una base de datos que ya existe en un volumen persistente. Hay que revisar las referencias a variables en Compose y aplicar la actualización de forma coherente en ambos servicios.

10. Conclusión

La práctica permite relacionar la teoría de imágenes y contenedores con una aplicación real. Docker Compose coordina la API y MySQL, mientras que las variables de entorno permiten ajustar la configuración sin modificar el código. La verificación no termina al ver los contenedores activos: también es necesario comprobar la salud de la base de datos, abrir Swagger y probar las operaciones de autenticación y consulta. Las evidencias reales son esenciales para demostrar que cada etapa se completó.

Referencias

Docker. (s. f.). *Docker Compose overview*. https://docs.docker.com/compose/

Docker. (s. f.). *docker compose down*. https://docs.docker.com/reference/cli/docker/compose/down/

Docker. (s. f.). *Docker CLI reference*. https://docs.docker.com/reference/cli/docker/

Carrenoutb. (s. f.). *biblioteca-api: README.md*. GitHub. https://github.com/carrenoutb/biblioteca-api/blob/main/README.md